

RANDY ZHU

Vancouver, BC | randy@randyzhu.com | github.com/RandoNandoz | linkedin.com/in/rzhu08 | Canadian Citizen · Open to Relocation

EDUCATION

University of British Columbia

April 2028

Bachelor of Science, Computer Science, Option in Software Engineering

GPA: 3.88/4

Relevant Coursework: Compilers, Programming Languages, Computer Architecture, Operating Systems

TECHNICAL SKILLS

Languages: C, C++, Python, Rust, TypeScript, Swift, Java, C#, SQL, Racket

Technologies: LVGL, SDL2, React, Embedded Linux, OpenWRT, LLVM IR, DriverKit, IOKit

Cloud/Infra: AWS (EC2, ECS), GCP (Cloud Run, Cloud SQL, Gemini), Cloudflare (DNS, Pages), Docker, Linux

Tools/Testing: Git, GitHub Actions, CMake, GDB, Valgrind, perf, Ghidra, pytest, Playwright, NUnit, JUnit

EXPERIENCE

Firmware Engineer Intern

May 2026 – Present

Netgear

Vancouver, BC

- Cut firmware RAM and eMMC footprint enough to eliminate a planned memory upgrade on Netgear's flagship Wi-Fi 7 router (~\$2M projected hardware cost avoided)
- Decreased runtime RAM usage by 400MB and eMMC usage by 600MB by replacing the WebKit-based LCD UI with a C-based, LVGL UI, freeing memory for additional product features
- Accelerated UI development by building a host-side build target that runs the firmware UI on a dev machine via SDL2, mocking the data engine and replacing the on-device fbdev/libinput/evdev backend, eliminating the UI flash-to-hardware test cycle
- Introduced clang-format and clang-tidy to the team, standardizing code style and adding static analysis to catch bugs before review

Research Intern

May 2025 – September 2025

Software Practices Lab

Vancouver, BC

- Carved fast pytest unit tests from slow end-to-end system tests, reducing runtime on large suites; identified assertion generation as state-based carving's core limit
- Built a Python unit test carving tool from scratch using dynamic execution tracing, AST transformations, and serialization to turn system tests into standalone, replayable unit tests
- Integrated an LLM (Gemma 3n) as a dependency classifier to auto-generate mocks for creating correct and deterministic unit tests

Software Developer Intern

September 2024 – April 2025

Teck Resources

Vancouver, BC

- Eliminated ~100 hours of manual data entry by building a Dataverse-API ingest tool in C#
- Introduced the team to Playwright + NUnit to automate end-to-end testing, **catching 87% of bugs before manual QA**
- Deployed a React.js-based calendar component across all Power Apps teams, implemented in TypeScript using the Fluent UI toolkit

Teaching Assistant (Introduction to Computer Systems)

July 2024 – Present

University of British Columbia

Vancouver, BC

- Awarded the **Undergraduate TA Award**, selected by a committee of faculty; achieved a 98% favourable rating on student experience of instruction surveys (n=102)
- Delivered a guest lecture on buffer overflow mitigations (canaries, ASLR, CFI and; covered professor's lectures to 200+ students

PROJECTS

macOS iSCSI Driver | C++, DriverKit, IOKit, Swift

November 2025 – Present

- Building an open-source macOS iSCSI initiator that mounts remote storage as a local disk, providing a free, auditable alternative to \$249/license proprietary tools
- Bridging the Swift userspace networking client to the driver extension over a lock-free shared-memory ring buffer, **saturating a 10Gb SFP+ NIC**, working around DriverKit's lack of network access
- Exposing the remote target as a local block device via the macOS IOUserSCSIParallelInterfaceController interface

Racket Compiler | x86_64 Assembly, LLVM IR, gdb, Racket

January 2026 – May 2026

- **Cut compile times 180x** (3h to 1m) by eliminating a pathological typed/untyped bottleneck in Typed Racket
- Replaced x86-only codegen with an LLVM IR backend, enabling compilation to any LLVM-supported platform
- Implemented closure conversion, tagged value representation, and tail call optimization via jump conversion

Sasquatch | Swift, ARKit, LiDAR, Detectron2, Open3D, Python, FastAPI, PostgreSQL, GCP

March 2026 (24h Hackathon)

- Deployed an ML pipeline combining Detectron2 instance segmentation and LiDAR point clouds to detect climbing wall holds
- Implemented PCA-based wall orientation detection to project 3D point clouds to 2D for inference
- **Reduced false positive rate from 90% to 1%** by adding a Gemini validation pass to filter non-hold wall objects